

Tarea 2 - Introducción a la Ciencia de Datos

Bruno Arnuti - Maximiliano Anzibar Fialho - 2025

1. Dataset y representación numérica de texto

1.1. Selección y Preprocesamiento de Datos

Para este trabajo se empleó un conjunto de datos reducido, compuesto exclusivamente por discursos de los tres candidatos con mayor número de registros: Joe Biden, Donald Trump y Mike Pence.

El primer paso consistió en una limpieza y preparación rigurosa del texto de los discursos. Esta tarea implicó remover etiquetas iniciales de los discursos (por ejemplo, "Speaker: (00:01)") para conservar únicamente el contenido relevante del discurso.

Adicionalmente, se consideró la presencia opcional de nombres propios de los candidatos mencionados en el contenido textual de cada discurso. Dado el interés por evaluar la influencia potencial de estas menciones en posteriores análisis predictivos, se definió un parámetro global configurable `INCLUDE_NAMES` que permite realizar fácilmente análisis tanto con como sin estos nombres. Si no se especifica lo contrario siempre asumimos que no se incluyen los nombres.

Finalmente, se aplicaron técnicas adicionales para homogeneizar el texto, tales como la eliminación de caracteres especiales, URLs, números aislados y signos de puntuación, además de una normalización general a minúsculas. El conjunto resultante de textos limpios se utilizó como base para las siguientes fases del análisis.

Para generar conjuntos representativos y balanceados, se realizó una partición estratificada de los datos, asignando un 30 % para pruebas (*test*) y el 70 % restante para desarrollo (*train*). La técnica de muestreo estratificado garantiza que la distribución proporcional de los discursos por candidato se mantenga constante en ambos subconjuntos, lo cual es fundamental para evitar sesgos en modelos predictivos posteriores. Además, para asegurar la reproducibilidad del experimento, se fijó el parámetro `random_state=1492`.

1.2. Visualización de Partición Estratificada del Conjunto

El tamaño final de los conjuntos resultantes fue de 100 discursos para entrenamiento y 43 para pruebas. En términos absolutos, el conjunto de entrenamiento quedó conformado por 37 discursos de Donald Trump, 50 de Joe Biden y 13 de Mike Pence, mientras que el conjunto de prueba incluyó 16 discursos de Donald Trump, 21 de Joe Biden y 6 de Mike Pence. En proporciones relativas, esto representa respectivamente el 37,0 %, 50,0 % y 13,0 % para el conjunto de entrenamiento, y el 37,2 %, 48,8 % y 14,0 % para el conjunto de pruebas.

La longitud promedio de los discursos fue de 6731 palabras (± 3891) en el conjunto de entrenamiento y 7393 palabras (± 4744) en el conjunto de prueba.

La estabilidad de la distribución puede apreciarse visualmente en la Figura 1, la cual muestra la distribución absoluta de discursos por candidato.

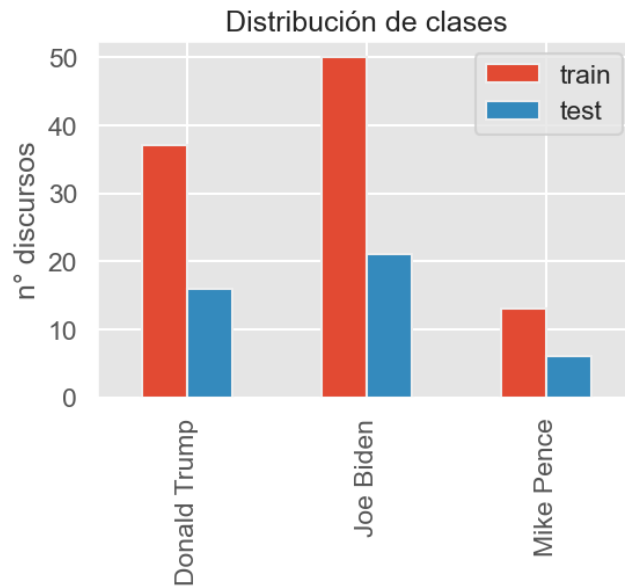


Figura 1: Distribución absoluta de discursos por candidato en los conjuntos de entrenamiento y prueba.

Para reforzar el análisis del balance de clases, se realizó una comparación porcentual ilustrada en la Figura 2. Adicionalmente, un test de chi-cuadrado de homogeneidad confirmó la ausencia de diferencias significativas entre ambos conjuntos ($p = 0,9855$), indicando un adecuado balance.

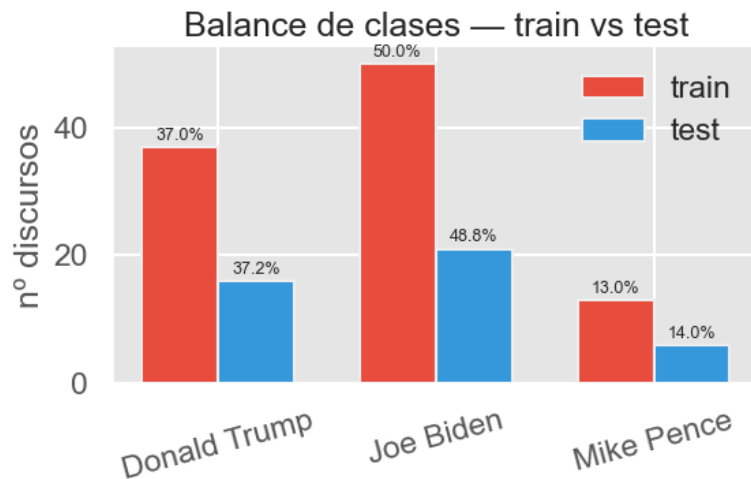


Figura 2: Comparación porcentual del balance de clases entre conjuntos de entrenamiento y prueba.

En resumen, el procedimiento aplicado para la preparación y partición del conjunto de datos resulta adecuado y confiable para análisis posteriores, garantizando que la evaluación de modelos predictivos no se vea comprometida por desequilibrios o sesgos en la distribución de los datos.

1.3. Representación numérica mediante Bag of Words

Para transformar los textos en representaciones numéricas, se empleó la técnica de *bag of words* (BoW). Esta técnica representa cada documento mediante un vector de conteos que refleja cuántas veces aparece cada palabra del vocabulario en dicho documento, sin considerar el orden o la estructura gramatical. Así, cada palabra única del corpus constituye una característica o *feature*.

En el conjunto de entrenamiento se obtuvo una matriz resultante de dimensiones 100×7783 , correspondiente a 100 discursos y un vocabulario total de 7783 términos distintos (7791 términos si se incluyen nombres propios en el texto). Un ejemplo ilustrativo es el primer discurso del conjunto de entrenamiento, en el cual destacan términos como ‘‘able’’ (21 veces), ‘‘speaker’’ (20 veces) y ‘‘schools’’ (18 veces).

La matriz generada es altamente dispersa (*sparse*), lo que implica que la mayoría de sus entradas son ceros, ya que cada discurso utiliza solo una pequeña fracción del vocabulario total disponible. En este caso específico, solo un 11,78 % (densidad) de las entradas tienen valores distintos de cero. Esta representación dispersa optimiza considerablemente el uso de memoria RAM. Por ejemplo, en una representación densa convencional esta matriz ocuparía aproximadamente 6,2 MB, mientras que en su formato disperso solo consume alrededor de 1,1 MB.

Si consideramos conjuntos de datos más grandes, típicos en problemas reales, el beneficio de esta representación dispersa sería aún más crucial, ya que una matriz densa requeriría cantidades prohibitivas de memoria, afectando severamente la capacidad computacional disponible.

Finalmente, las Figuras 3 y 4 presentan un análisis comparativo de los veinte términos más frecuentes en el conjunto de entrenamiento, tanto excluyendo como incluyendo los nombres propios de los candidatos. Esto permite observar claramente el impacto que tiene esta decisión sobre la frecuencia relativa de los términos.

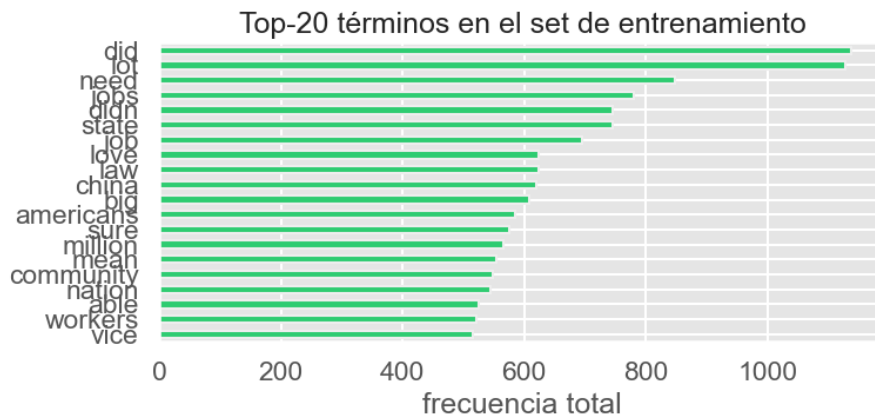


Figura 3: Top-20 términos más frecuentes en el conjunto de entrenamiento sin nombres propios.

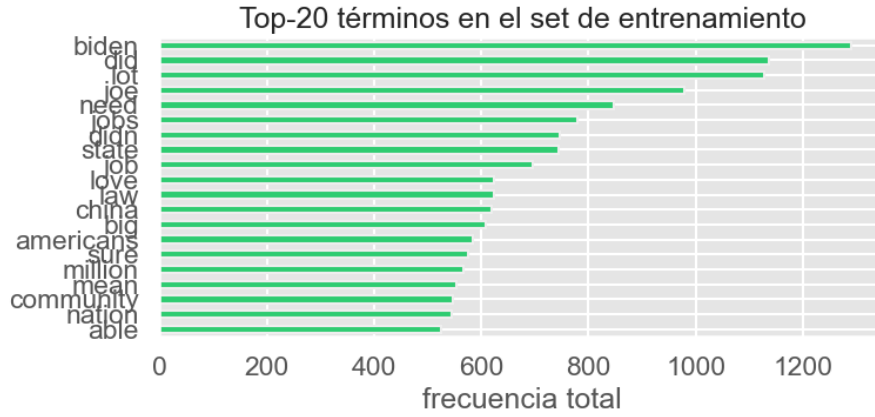


Figura 4: Top-20 términos más frecuentes en el conjunto de entrenamiento incluyendo nombres propios.

1.4. Representación TF-IDF y análisis de n-gramas

Un *n-grama* es una secuencia contigua de n palabras. Por ejemplo, un *unigrama* ($n = 1$) representa una sola palabra; un *bigrama* ($n = 2$) representa un par de palabras consecutivas; y un *trigrama* ($n = 3$) representa una secuencia de tres palabras consecutivas. Estos modelos permiten capturar patrones de coocurrencia y dependencias locales en el lenguaje, conocidas como *dependencias de corto alcance*, que son fundamentales para entender el contexto en el que aparece una palabra.

Modelos de este tipo se basan en una simplificación estadística del lenguaje conocida como cadenas de Markov, donde se asume que la probabilidad de una palabra depende únicamente de un número fijo de palabras anteriores. Por ejemplo, en una cadena de primer orden (como un modelo de bigrama), cada palabra depende solo de la palabra anterior.

El modelo más simple es el de unigrama, que asume que cada palabra aparece de forma independiente de las demás. Su probabilidad se estima como:

$$P(w_i) = \frac{\text{Frecuencia del término } w_i}{\text{Cantidad total de palabras en el corpus}}$$

Este enfoque ignora por completo el contexto en el que aparece la palabra. Para mejorar esta limitación, los modelos de bigrama incorporan la dependencia con la palabra inmediatamente anterior, estimando la probabilidad condicional de que una palabra aparezca dada su predecesora:

$$P(w_i | w_{i-1}) = \frac{\text{Frecuencia de la secuencia } w_{i-1}w_i}{\text{Frecuencia de } w_{i-1}}$$

Aún más ricos son los modelos de trigrama, que consideran las dos palabras anteriores, capturando así una dependencia de segundo orden:

$$P(w_i | w_{i-2}, w_{i-1}) = \frac{\text{Frecuencia de la secuencia } w_{i-2}w_{i-1}w_i}{\text{Frecuencia de la secuencia } w_{i-2}w_{i-1}}$$

De forma general, un modelo n -grama estima la probabilidad de una secuencia de palabras $w_1, w_2, \dots, w_T$ como el producto de las probabilidades condicionales de cada palabra dada las $n - 1$ anteriores:

$$P(w_1, \dots, w_T) \approx \prod_{t=1}^T P(w_t \mid w_{t-n+1}, \dots, w_{t-1})$$

Si bien su simplicidad los hace accesibles y útiles para tareas como clasificación o recuperación de información, también presentan limitaciones para modelar dependencias a largo plazo.

La representación numérica adicional utilizada fue *Term Frequency-Inverse Document Frequency* (TF-IDF). Esta técnica combina la frecuencia de los términos en cada documento (TF) con la frecuencia inversa en la colección (IDF), penalizando así los términos muy frecuentes que aportan poca capacidad de discriminación entre documentos. Matemáticamente, un término frecuente en un documento pero poco frecuente en el corpus completo tendrá un valor TF-IDF alto, destacando así su importancia relativa.

La matriz TF-IDF generada para el conjunto de entrenamiento posee dimensiones 100×37374 (o 100×38182 si se incluyen los nombres propios), con un 94,6 % de entradas igual a cero, lo que indica nuevamente una alta dispersión. Esta dispersión implica un considerable ahorro de memoria: mientras una representación densa ocuparía aproximadamente 29,9 MB, la versión dispersa utiliza solamente 2,40 MB, lo que representa un ahorro de recursos computacionales de más de 12 veces.

Un análisis de los términos con mayor peso promedio en la matriz TF-IDF revela elementos en los discursos, como se muestra en la Figura 5 (sin nombres propios) y en la Figura 6 (con nombres propios). Se observa cómo la inclusión de nombres personales afecta ligeramente la relevancia relativa de algunos términos y bigramas, introduciendo explícitamente nombres propios como factores importantes.

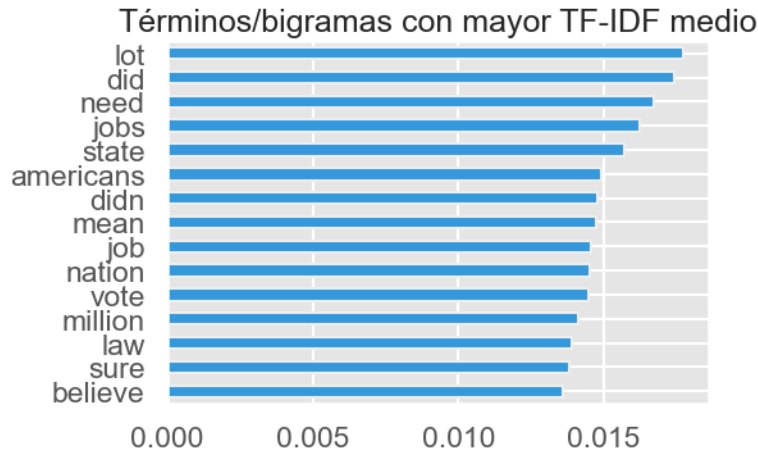


Figura 5: Términos y bigramas con mayor TF-IDF promedio en el conjunto de entrenamiento (sin nombres propios).

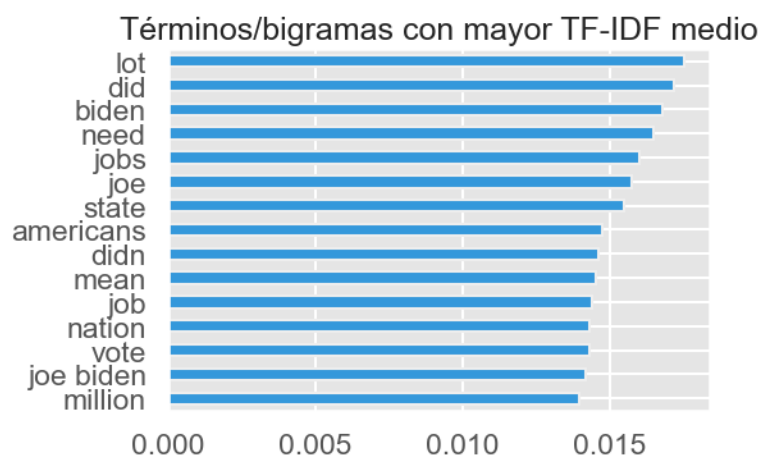


Figura 6: Términos y bigramas con mayor TF-IDF promedio en el conjunto de entrenamiento (incluyendo nombres propios).

Este análisis pone en evidencia cómo la combinación de TF-IDF y n-gramas permite identificar términos significativos específicos del contexto, mejorando la eficacia en tareas de clasificación o recuperación de información.

1.5. Análisis y comparación con PCA y SVD sobre vectores TF-IDF

Para analizar la estructura interna del conjunto de entrenamiento transformado mediante TF-IDF, se recurrió a técnicas de reducción de dimensionalidad como PCA (Análisis de Componentes Principales) y SVD (Descomposición en Valores Singulares). Estas técnicas permiten proyectar datos altamente dimensionales en un espacio reducido de pocas dimensiones preservando, en la medida de lo posible, la varianza original.

Primeramente, se aplicó PCA sobre los vectores densificados de TF-IDF. La visualización utilizando las dos primeras componentes principales se presenta en la Figura 7. Observamos que, aunque las dos primeras componentes principales explican solamente el 7,52% de la varianza total, existe una separación parcial pero notable entre los discursos de los candidatos Donald Trump, Joe Biden y Mike Pence. Esto implica que estas componentes capturan patrones lingüísticos relativamente característicos de cada orador, siendo Trump y Biden particularmente distinguibles, mientras Pence aparece algo más separado en la proyección.

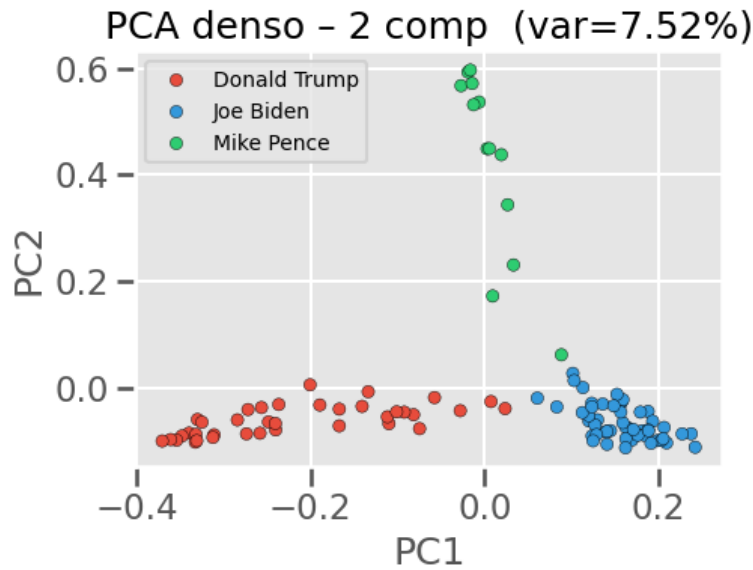


Figura 7: Mapa 2D de discursos según las dos primeras componentes PCA sobre vectores TF-IDF densificados.

Adicionalmente, se analizó cómo evoluciona la cantidad de varianza explicada conforme aumenta el número de componentes PCA (Figura 8). Claramente, aunque la adición sucesiva de componentes incrementa la varianza explicada, esta acumulación es relativamente lenta. Incluso con 10 componentes, la varianza acumulada apenas supera el 20 %, indicando la alta dimensionalidad y complejidad del espacio original generado por TF-IDF.

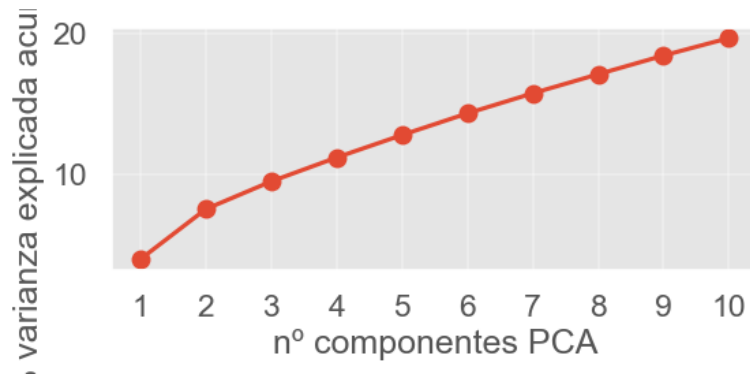


Figura 8: Varianza acumulada explicada por componentes principales PCA sobre vectores TF-IDF.

Complementariamente, se realizaron análisis adicionales mediante SVD truncado sobre distintas configuraciones del vectorizador TF-IDF, comparando escenarios con y sin filtrado de *stop-words*, y variando el parámetro IDF y el rango de n-gramas. La Figura 9 expone claramente las diferencias entre configuraciones. Notablemente, la configuración sin utilizar IDF (configuración C) alcanzó una varianza explicada superior (7.29 %) en sus dos primeras componentes, en contraste con aquellas que sí aplican IDF (4.47 % y 4.68 % respectivamente en configuraciones A y B). Este resultado sugiere que la inclusión del término inverso de frecuencia documental, aunque es útil para ciertos análisis predictivos, reduce la varianza explicada al transformar el espacio en uno menos disperso y más normalizado.

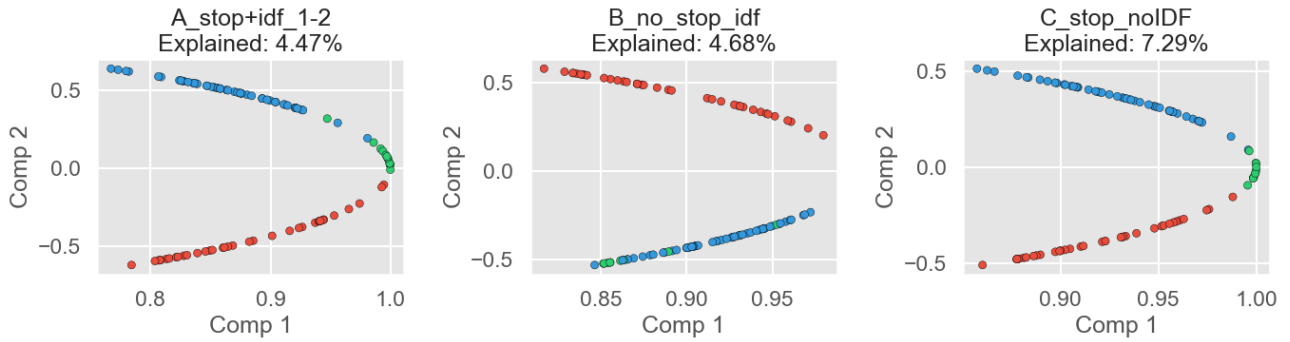


Figura 9: Proyección 2D con SVD truncado para diferentes configuraciones del vectorizador TF-IDF.

Finalmente, al analizar la curva acumulada de varianza explicada por SVD truncado (Figura 10), observamos un patrón similar al del PCA denso. La acumulación de varianza explicada es relativamente lenta y limitada en pocas dimensiones, reflejando nuevamente la dificultad inherente de capturar eficientemente la riqueza léxica de discursos políticos con un reducido número de componentes.

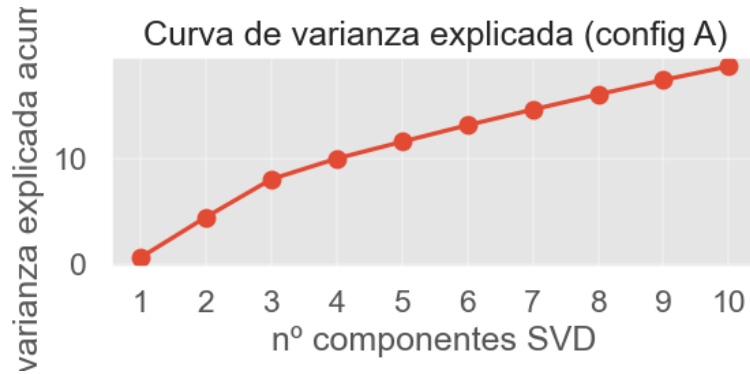


Figura 10: Curva de varianza explicada acumulada según componentes del SVD truncado (configuración A: stop-words e IDF activados, n-gramas de 1 y 2 palabras).

En conclusión, aunque la reducción de dimensionalidad mediante PCA y SVD logra detectar ciertos patrones lingüísticos distintivos en los discursos, la limitada cantidad de varianza capturada en pocas dimensiones refleja claramente la complejidad intrínseca del corpus estudiado. Esto indica que, para un análisis más robusto y predictivo, probablemente se requiera considerar un número mayor de dimensiones o complementar el análisis con técnicas adicionales que capturen aspectos semánticos o contextuales del lenguaje.

No se notan cambios significativos al agregar nombres, por lo que no se agregan dichas gráficas.

2. Entrenamiento y Evaluación de Modelos

En esta segunda etapa utilizaremos los datos de palabras tokenizadas para entrenar un modelo que permita predecir a qué candidato corresponde el conjunto de palabras asociado a un nuevo discurso.

2.1. Clasificar con Modelo Naive Bayes

En primer lugar utilizamos un modelo de clasificación Multinomial Naive-Bayes, el cual es ampliamente utilizado en clasificación de texto. Este algoritmo funciona considerando que cada feature es independiente de los demás aunque pertenezcan a la misma categoría, es decir, en este caso, es como si cada palabra en el discurso es generada independientemente de las demás. Esta suposición facilita y acelera los cálculos, lo que lo hace un modelo simple, necesitando pocos datos de entrenamiento para estimar los parámetros de clasificación, sin embargo al ser un modelo bastante básico no suele tener un alta performance.

Se entrenó el modelo con el valor por default del hiperparámetro $\alpha=1$. Este parámetro corresponde a un suavizado en los cálculos de probabilidades bayesianas que realiza el modelo. El valor default deja inalterado estos cálculos, un valor menor a 1 corresponde a un suavizado poco agresivo de las funciones de distribución de probabilidad y un valor mayor a 1 corresponde a un suavizado más agresivo (haciendo tender las distribuciones a una uniforme por lo tanto reduciendo la influencia de los features/palabras de alta frecuencia).

Luego de entrenar se aplicó el modelo al conjunto de datos de test obteniendo un valor de *accuracy* = 0,93. Esta métrica es una medida global de la performance del modelo y corresponde al ratio entre predicciones correctas sobre el total de predicciones. La principal desventaja de esta métrica es que no discrimina por clase, es decir que al tratar todas las clases con la misma importancia no permite ver si estamos teniendo una mala predicción en una de ellas o en varias. Además en los casos donde hay un gran desbalance de clases esto se acentúa, ya que por ejemplo podemos estar prediciendo bien una clase muy numerosa pero no relevante, y en ese caso tendríamos un *accuracy* alto, mientras que la clase objetivo o relevante puede estar mal clasificada la mayoría de las veces.

Por lo tanto, para visualizar cómo se distribuyeron las predicciones correctas e incorrectas en cada clase se grafica una matriz de confusión (Figura.11). En la misma se puede observar que para Biden todas las predicciones fueron correctas, para Trump hubo 14 correctas y 2 incorrectas clasificadas como correspondientes a Biden, y finalmente para Pence hubo 5 correctas y 1 incorrecta predicha como correspondiente a Biden. Asociado a esto podemos reportar los valores de *recall* y *precision* para cada candidato como se muestra en la tabla a continuación.

Candidato	Precision	Recall
Trump	1.000	0.875
Biden	0.875	1.000
Pence	1.000	0.833

La *precision* mide la cantidad de positivos verdaderos en función de todas las instancias que se clasificaron como verdaderas. Por ejemplo, para Trump vemos que la precision es 1 porque de todas las instancias clasificadas como "Trump" todas de ellas fueron de datos correspondientes Trump, es decir su etiqueta original era Trump. De la mismas forma para Pence. En cambio para Biden la precisión es 0.875 porque de todas las instancias clasificadas como Biden (24 en total) solo 21 correspondían a la etiqueta Biden, mientras que 2 correspondían a la etiqueta Trump y 1 a Mike Pence. En la matriz de confusion esto lo podemos ver mirando por columna y evaluando si la única entrada es en la diagonal o hay elementos por fuera de la diagonal. Para las columnas de Trump y Pence solo hay elementos en la diagonal por lo tanto la precisión es 1. Esta métrica es útil cuando el costo de los falsos positivos es alto, es decir cuando queremos medir con seguridad que todas las predicciones positivas corresponden a su clase. La desventaja de esta métrica es que no mide los falsos negativos, es decir que no nos permite evaluar cuando clasificamos mal un candidato.

Por otro lado, la métrica de *recall* nos indica la cantidad de clasificaciones correctas para una clase en función de todas las clasificaciones posibles para dicha clase. Es decir, en nuestro ejemplo, es la

cantidad de veces que se clasificó correctamente a un candidato en función de todos los discursos correspondientes a dicho candidato. En la matriz de confusión esto lo podemos observar si miramos la información por filas. Por ejemplo, para Trump había 16 discursos pero 14 fueron etiquetados correctamente y 2 no, por lo tanto la *recall* corresponde a 0.875 ($=14/16$). Lo mismo ocurre para Pence donde de los 6 discursos etiquetados con su nombre 5 se clasifican correctamente, mientras que para Biden el *recall* = 1 ya que todos los discursos se clasificaron correctamente. Esta métrica es buena cuando queremos determinar clases con pocos falsos negativos, es decir que todos los elementos de esa clase se clasifiquen correctamente. Además, al igual que para la precisión, estas métricas son útiles en el caso de trabajar con datos muy desbalanceados en las clases, ya que permiten obtener información específica de la clasificación de cada clase. La desventaja del *recall* es que no toma en cuenta los falsos positivos, es decir que poder encontrar discursos clasificados para una clase que no es la correspondiente.

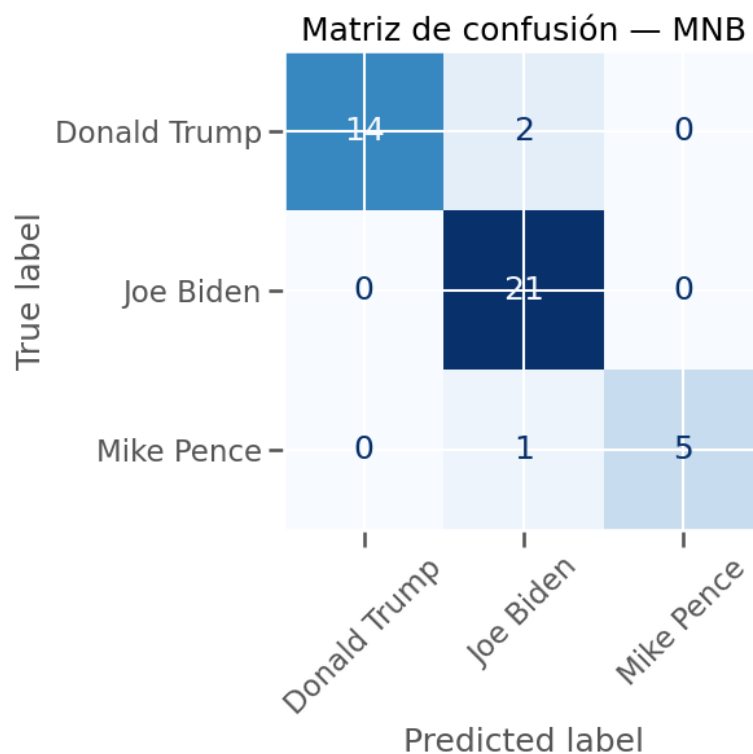


Figura 11: Matriz de confusión para la predicción del conjunto de datos test utilizando el modelo Multinomial Naive-Bayes. La matriz muestra la relación entre la etiqueta real de los datos (filas) y la etiqueta predicha por el modelo (columnas). Se puede observar que hubo 40 predicciones correctas en total y 3 incorrectas, dos discursos de Trump clasificados como Biden y uno de Pence clasificado como Biden.

2.2. Validación cruzada y elección de hiperparámetros

El procedimiento más simple que se puede utilizar para optimizar un modelo sería separar el conjunto de datos en un set de entrenamiento y un set de testeo. Luego variar los hiperparámetros de forma de tener distintas versiones de modelos entrenados y quedarse con el que tenga mejor rendimiento en el set de testeo. Sin embargo, esto podría generar que el modelo se este sobreajustando a los datos de test y no sea necesariamente un buen modelo para predecir otros datos por fuera del set de test. Una forma de abordar esto es separando los datos originales en 3 particiones: entrenamiento, validación y testeo. En este caso se introduce un nuevo conjunto, el de validación, que tiene como objetivo evaluar al modelo mientras está siendo entrenado de forma de ir obteniendo los parámetros/pesos que mejor

generalicen al conjunto de validación, y luego una vez finalizado este proceso se pasa a evaluar en el set de testeo. Esto es una práctica muy común en los procesos de entrenamiento, sin embargo, en los conjuntos de datos reducidos esto puede representar un problema ya que tendríamos muy poca cantidad de datos para entrenar y validar. Una alternativa a esto es la técnica de validación cruzada o *cross-validation*.

Un esquema de cómo funciona esta técnica se puede observar en la figura 12. Partimos nuevamente el total de los datos en un conjunto de entrenamiento y un conjunto de testeo, y luego el set de entrenamiento se separa en k particiones de igual tamaño (llamadas Folds), en el ejemplo de la figura $k = 5$. Luego, en un primer paso, se utilizan $k - 1$ particiones para realizar el entenamiento y se ajustan los parámetros evaluando las métricas en la partición restante. De forma iterativa se aplica este mismo procedimiento pero dejando fuera del entrenamiento siempre una partición distinta. Finalmente luego de ajustar los parámetros para todas las subparticiones del set de entrenamiento se procede a probar el modelo en el set de testeo. Las métricas generales del entrenamiento resultan un promedio de todas las iteraciones realizadas para cada subpartición.

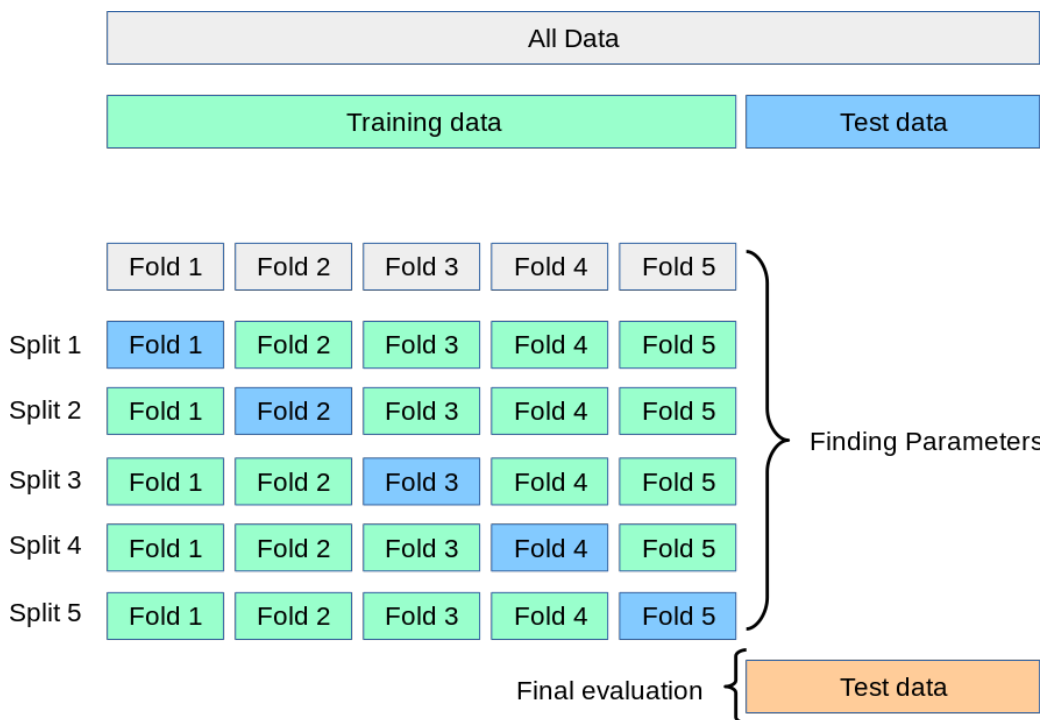


Figura 12: Esquema de la técnica de validación cruzada.

En nuestro caso, implementamos el método de validación cruzada utilizando el módulo *GridSearchCV* de Scikit-learn, que nos permitirá elegir los mejores hiperparámetros para nuestro modelo. El nombre de la función proviene de que toma como entrada una "grilla" de hiperparámetros que nosotros elegimos y genera métricas para cada una de las distintas combinaciones de hiperparámetros en la grilla. Por lo tanto los principales inputs de la función *GridSearchCV* son: el tipo de modelo que queremos entrenar, la grilla de hiperparámetros, y el número de subparticiones para la validación cruzada.

El módulo *GridSearchCV* también permite pasar como entrada un pipeline de procesamiento en vez de únicamente el modelo, de esta forma se pueden poner dentro de la grilla de hiperparámetros algunas de las propiedades del pipeline, permitiendo entrenar modelos para distintos procesamiento de los datos. En nuestro caso generamos un pipeline para obtener los vectores *tf-idf* y luego aplicar el modelo Naive-Bayes. Por lo tanto dos de los parámetros que utilizamos en la grilla corresponden a la vectorización de los discursos, estos son: 1) rango de n-grama, que puede ser 1 o 2, es decir si se consideran palabras individuales o combinación de dos palabras en la tokenización, esto puede ayudar a capturar más el contexto pero es costoso computacionalmente; 2) la mínima frecuencia de palabra, tomando valores 1 o

2, es decir incluir todos los términos que aparezcan 1 vez en cualquier documento o excluir los términos que aparecen 1 vez en solo 1 documento. El tercer hiperparámetro para GridSearchCV es el valor de *alpha* para el modelo Naive-Bayes para el cual colocamos valores [0.1, 1.0, 5.0]. En la figura 13 podemos observar los resultados de los valores promedio de accuracy para cada combinación de hiperparámetros y tomando una subpartición de 5 para la validación cruzada. Como se puede ver el valor que influye realmente en la performance del modelo es el *alpha* ya que para *alpha* = 0,1 obtenemos la mejor configuración independientemente de los otros hiperparámetros. Finalmente, la mejor combinación de hiperparámetros termina siendo: [*alpha* = 0,1, *min_freq* = 2, *n_gram_range* = 1].

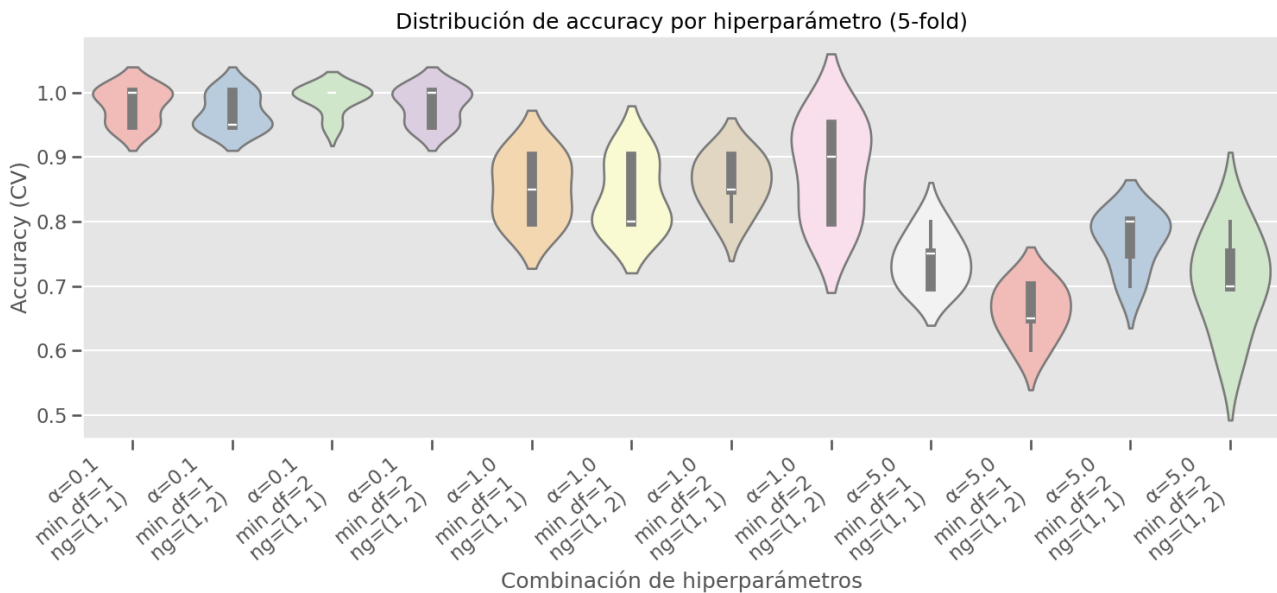


Figura 13: Valor de accuracy del modelo Naive-Bayes para distintos hiperparámetros evaluados utilizando GridSearchCV

En base a los mejores parámetros obtenidos se volvió a entrenar el modelo utilizando todos los datos de train (es decir, se utilizó una única partición entre train y test, sin utilizar datos para validación). En este caso se obtuvo un valor de *accuracy* = 1,0 y la matriz de confusión que se muestra en la figura 14. Como se puede observar los valores de recall y precision serían 1 para todos los candidatos ya que todas las instancias fueron predichas correctamente. Comparando con el modelo por default observamos que aplicar GridSearch permitió mejorar el modelo, obteniendo mejores métricas. El hecho de que el valor de accuracy sea 1.0 no es muy realista para modelos grandes donde hay mucha variabilidad, en este caso al ser un modelo bastante pequeños parecer ser fácil la tarea de separar entre estas tres clases. De esta forma concluimos que el procesamiento de texto utilizando las técnicas de *bag of words* y *tf.idf* resulta efectivo en este caso y es una muy buena aproximación para modelos de análisis de lenguaje pequeños. Sin embargo, algunas de las limitaciones que tiene este procesamiento son:

- La falta de contexto o semántica ya que se ignora el orden de las palabras así como la relación entre ellas.
- Alta dimensionalidad, ya que cada palabras es un feature y por lo tanto terminamos obteniendo matrices de muy esparsas incrementando el costo computacional.
- Falta de manejo de sinónimos, ya que dos palabras con el mismo significado representan dos features distintos, lo cual también adhiere dimensionalidad innecesaria al problema.
- El espacio de representación son vectores fijos, es decir que no pueden cambiar su tamaño de forma dinámica. Una vez que se estableció el vocabulario de todas las palabras este no puede cambiarse.

En la actualidad alguno de los modelos más avanzados de procesamiento de texto atacan estas falencias para generar representaciones más eficientes y con más significado, algunos de ellos son BERT, Word2vect o transformers.

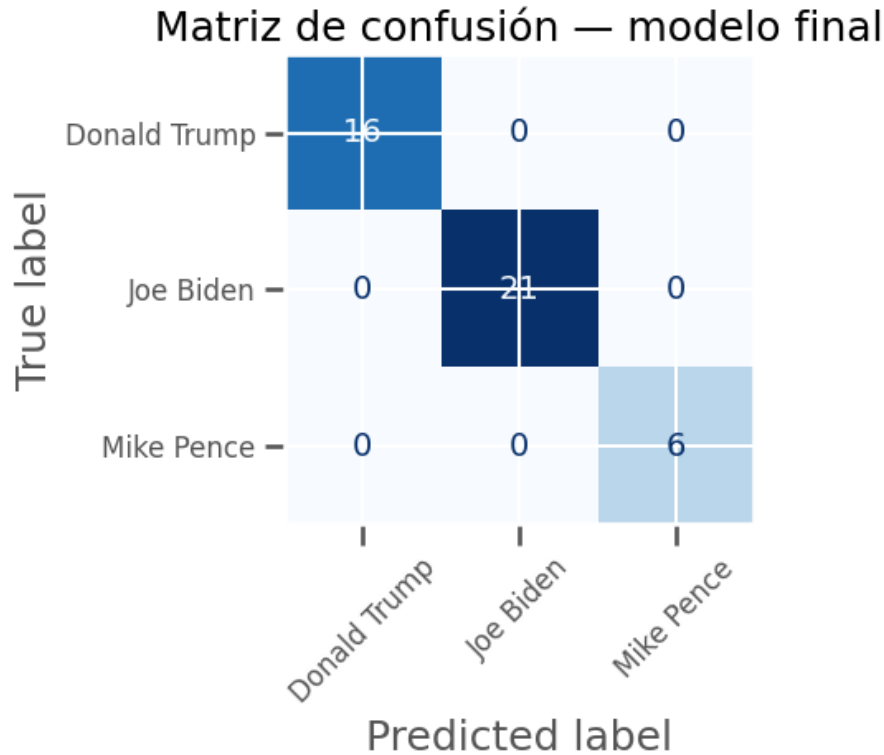


Figura 14: Matriz de confusión para el mejor modelo obtenido a partir de los mejores parámetros del GridSearchCV.

2.3. Modelo alternativo para clasificación

Como modelo alternativo para clasificar los discursos utilizamos la **regresión logística**, el cual es ampliamente utilizado para las tareas de clasificación. La forma binaria de la regresión logística funciona modelando la probabilidad de obtener una clase positiva según la función sigmoide:

$$p(X, b, w) = \frac{1}{1 + e^{-w \cdot X + b}} \quad (1)$$

donde X es la variable, w y b los parámetros de ajuste, el peso y bias respectivamente. En función de la curva sigmoide se decide un threshold sobre el cual la clase va a ser clasificada como positiva (1 por ejemplo) o negativa (cero). Generalmente el threshold suele ser 0.5.

Para el caso de la clasificación multiclase la distribución de probabilidad es una extensión de lo anterior y pasa a ser lo que se conoce como función *softmax*:

$$softmax(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (2)$$

donde $z_i = w_i \cdot X + b_i$ y K es la cantidad de clases. La salida de esta función es por lo tanto la probabilidad de que el input pertenezca a cada clase, luego se toma como predicción la clase con

mayor probabilidad.

En nuestro caso, entrenamos el modelo de regresión logística con los valores por default ($C=1.0$, $\text{penalty}=l2$) y obtuvimos un $\text{accuracy} = 1.0$. La matriz de confusión resultante es por lo tanto análoga a la de la figura 14 ya que todas las predicciones fueron correctas (ver código asociado).

2.4. Introducción de nuevo candidato

Finalmente, el último análisis que realizamos fue cambiar uno de los candidatos para evaluar cómo cambia la eficacia de la clasificación. En este caso se retiró el candidato con menos discursos, Mike Pence, y se agregó la candidato Kamala Harris, siguiente en cantidad de discursos. Volvimos a entrenar el modelo Naive-Bayes con los mismos parámetros óptimos obtenidos del GridSearch, y alcanzamos un valor de $\text{accuracy} = 0.93$. En este caso los valores de precision y recall fueron los siguientes:

Candidato	Precision	Recall
Trump	1.00	1.00
Biden	0.88	1.00
Kamala	0.00	0.00

que se relacionan con la siguiente matriz de confusión (figura 15). Como se puede observar los discursos de Trump y Biden son bien clasificados, mientras que los de Harris son, en todos los casos, clasificados como Biden. Esto puede tener algo de sentido en función de que Harris fue compañera de fórmula de Biden. En particular si los discursos corresponden al final de la campaña hay más probabilidad de que tengan similitudes con los de Biden: para eliminar este sesgo se recomendaría mezclar aleatoriamente los discursos para que los sets de train y test tengan una buena representación temporal de cuándo se obtienen los datos.

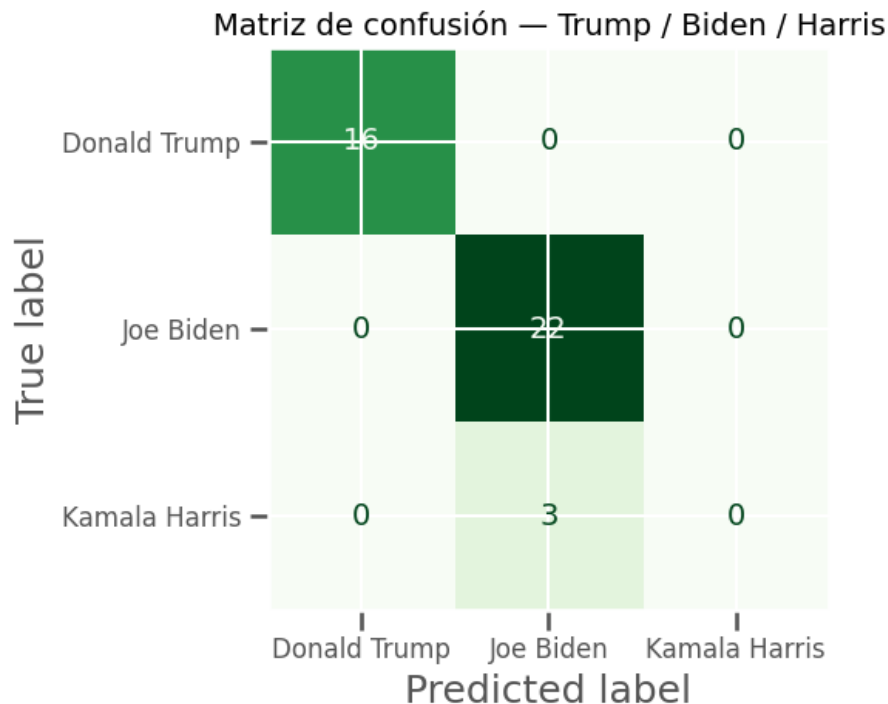


Figura 15: Matriz de confusión para el modelo Naive-Bayes entrenado para los candidatos Trump-Biden-Harris.

2.5. Alternativas para la extracción de features de texto

Como se mencionó más arriba, el procesamiento de texto utilizando *bag of words* y *tf-idf* contiene ciertas limitaciones y existen alternativas que resuelven algunas de ellas. La principal limitaciones es tal vez la falta de contexto o relación entre las palabras, ya que tratarlas como elementos aislados no aporta mucha información a lo que se está diciendo con esas palabras. La piedra clave para las técnicas más avanzadas de procesamiento de texto son los *embeddings*. Estos son representaciones que permiten capturar el contexto y la similaridad de las palabras, y se logran mapeando las palabras a un espacio de representación donde palabras similares o relacionadas conceptualmente se encuentren cercanas en dicho espacio. Este espacio de representación es un espacio continuo de números reales, luego cada palabras se codifica en un vector, que tiene la dimensión de este espacio, en función de su significado. De esta forma palabras similares en contexto, por ejemplo rey y reina, van a tener vectores similares. Cada entrada de este vector corresponde a un feature que no tiene asignado un significado específico a priori, pero imaginemos que una de estas entradas del vector representa un atributo similar a realeza, entonces las palabras rey y reina tendrán un valor alto en dicha entrada del vector, mientras que otras palabras como perro o árbol tendrán valores muy bajos. Así cada entrada del vector (cada dimensión del espacio de embeddings) contiene un significado o atributo, no determinado, que va a permitir relacionar palabras con contextos similares. Una de las grandes ventajas adicionales de este método de codificación es que la dimensión del espacio es mucho menor que al utilizar el *bag of words*, ya que no necesitamos una dimensión por cada palabra sino que necesitamos un número de dimensiones suficiente que permita agrupar palabras según su contexto, lo cual suele ser de muchos órdenes menor a la cantidad de palabras únicas que aparecen en un texto.

Explicaremos brevemente uno de los modelos que permite realizar estos embeddings, en este caso uno de los más utilizados, **Word2vect**. Como sugiere su nombre lo que el modelo busca es llevar una palabra al formato de vector en el espacio de representaciones tal como se describió. La técnica consisten en una red neuronal pequeña que va a tener 3 capas:

1. la **capa de entrada o input**, esto va a ser un vector de alta dimensionalidad (N) que contenga, al igual que en *bag of words*, una codificación tipo one-hot para cada palabra que aparezca en el texto, por lo tanto N es la dimensión del vocabulario (el número total de palabras distintas en el texto);
2. la **capa oculta o vector de embedding**, esa será una capa de neuronas de baja dimensionalidad que mapea la capa de entrada a la salida del modelo. Los pesos de esta capa serán las representaciones aprendidas para llevar las palabras a un vector de baja dimensionalidad que capture el contexto de la palabra. Por lo tanto, la dimensión el vector de embeddings se corresponde con la cantidad de neuronas (n) de este espacio.
3. la **capa de salida o output**, es una capa de igual dimensión que la entrada, N , que contiene la probabilidad (a partir de una función softmax) de que un input represente una palabra del vocabulario. Luego se toma la entrada de máxima probabilidad para seleccionar la palabra.

Por lo tanto este modelo de red neuronal se utiliza para extraer los pesos de la capa oculta que corresponderán justamente al embedding en el espacio de representación pequeño (n).

Existen entonces 2 formas en que la técnica de Word2vect puede extraer estos embeddings. El primero de ellos se llama **Continuous bag-of-words** (CBOW), y consiste en predecir una palabra en función de las palabras que la rodean en la frase. Por ejemplo, en la frase "la torta es de sabor chocolate", el modelo tomará como entrada un vector que codifique las palabras 'la', 'es', 'de', 'sabor', y 'chocolate' y ajuste los pesos de la capa oculta para predecir como salida 'torta'. De esta forma el concepto torta queda embebido en el contexto de las otras palabras. El modelo prueba con distintas palabras para

que aquellas que tengan un significado similar (por ejemplo, 'torta' o 'galleta') tengan una relación cercana en el espacio de representaciones, es decir que influyen las probabilidades de salida de forma similar. El segundo método de embedding se denomina **Skipgram**, el cual funciona de forma inversa al CBOW. En este caso el input es una única palabra y se intenta predecir la probabilidad de que otras palabras estén cercanas, es decir busca aprender el contexto del entorno de dicha palabra. Esto se hace tomando una ventana de palabras entorno a la elegida y se toman combinaciones distintas de dicha ventana, por ejemplo para una ventana de largo 2 sería: $palabra_elegida - 1$ y $palabra_elegida + 1$, esto se toma entonces como el target de salida del modelo y se ajustan los pesos para entrenar según este target. En este caso Skipgram tiene un mayor costo computacional pero funciona mejor para codificar palabras poco frecuentes en el texto.